

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

Debugging or Optimization task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

3/8



P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Producer–Consumer Debugging (Semaphore Error)

A producer-consumer application occasionally hangs after running for several minutes.

Semaphore empty = 5
Semaphore full = 0
Semaphore mutex = 1

Producer:

```
wait(mutex)
wait(empty)
insert(item)
signal(full)
signal(mutex)
```

Consumer:

```
wait(mutex)
wait(full)
remove(item)
signal(empty)
signal(mutex)
```

The program freezes even though both producer and consumer threads are alive.

1. Identify the synchronization error causing the hang.
2. Explain why the semaphore ordering is incorrect.
3. Suggest the correct semaphore sequence to eliminate the problem.

(10 Marks)

Q7. Thread Performance Debugging

A web server creates one new thread for every incoming client request.

Performance statistics:

- Average requests/sec = 15,000
- Average thread creation time = 0.4 ms
- CPU utilisation = 92%
- Memory usage continuously increases

Developers notice that the server becomes slow after several hours.

1. Identify the thread-management problem.
2. Explain why the current multithreading model causes performance degradation.
3. Propose two optimisations to improve scalability.



(10 Marks)

Q8. Banker's Algorithm Debugging

Available Resources:

A B C = 2 1 1

Allocation:

P1: 1 0 1

P2: 0 1 0

P3: 1 1 1

Maximum:

P1: 2 1 2

P2: 1 2 1

P3: 3 2 2

Process P2 requests (1,1,1).

1. Calculate the Need matrix.
2. Determine whether the request can be granted immediately using Banker's Algorithm.
3. If not, identify the unsafe condition and explain why granting the request could lead to deadlock.

(10 Marks)

Q9. Deadlock Detection and Recovery Debugging

A database server reports that four transactions are waiting for resources.

Resource Allocation Graph:

- T1 waits for R2
- T2 waits for R3
- T3 waits for R4
- T4 waits for R1
- R1 allocated to T1
- R2 allocated to T2
- R3 allocated to T3
- R4 allocated to T4

The database becomes completely unresponsive.



1. Identify whether a deadlock exists.
2. Explain how deadlock detection identifies this situation.
3. Suggest two recovery techniques and justify which one minimises system disruption.

(10 Marks)

Q10. Monitor-Based Synchronization Debugging

A monitor is used to synchronize access to a shared printer.

```
monitor Printer
{
    boolean busy = false;

    procedure Print()
    {
        if(busy)
            wait();

        busy = true;

        // Printing

        busy = false;
    }
}
```

Users report that some print jobs never execute even though the printer becomes free.

1. Identify the logical error in the monitor implementation.
2. Explain why waiting processes are never awakened.
3. Modify the monitor logic to ensure correct synchronization and fairness.

(10 Marks)

Rubrics for Calculation



CO2 AT 3

Debugging

B. Venkata Uday

Kumar

192411203

CSA0404

Operating system.

Q1 Deadlock Debugging

Root cause of Deadlock :- The deadlock occurs because the two processes acquire shared resources in opposite order.

$P_1: R_1 \rightarrow R_2$

$P_2: R_2 \rightarrow R_1$

Execution sequence

1. P_1 locks R_1 .

2. P_2 locks R_2 .

3. P_1 waits for R_2 .

4. P_2 waits for R_1 .

Both processes wait forever.

Deadlock condition

* Mutual Exclusion

* No preemption

* Hold and wait

* No preemption.

2. Optimisation

Use resource ordering

Process P_1

lock(R_1)

lock(R_2)

critical section

unlock(R_2)

unlock(R_1)

Process P_2

lock(R_1)

lock(R_2)

critical section

unlock(R_2)

unlock(R_1)

Alternative optimisation

* try-lock with timeout

* deadlock detection and recovery.

Q2 CPU utilisation Problem

Root cause

Processes continuously move

Ready $\rightarrow$ waiting $\rightarrow$ Ready $\rightarrow$ waiting.

$\rightarrow$ This indicates frequent I/O blocking.

Thus CPU utilisation drops to 30%, although runnable processes exist.

Debugging Analysis

The workload is I/O-bound, not CPU-bound.

Frequent context switching and I/O waits reduce processor efficiency.

System-level optimisation.

1. Increase degree of multiprogramming.
2. Improve disk scheduling (SCAN, C-SCAN).
3. Use DMA for data transfer.

Q3 Memory Bottleneck Debugging

Given

code = 5 MB Heap = 150 MB

stack = 1 MB $\times$ 25 threads

Total Memory used

$$150 + 25 + 5 = 180 \text{ MB}$$

Although RAM is 2 GB, the application experiences excessive page faults.

Root cause

The heap allocation is large and shared among many threads.

- * Poor cache locality
- * Increased page faults.
- * Thrashing.

Optimisation 1

Reduce heap usage,

- * Release unused objects.

- * Use object pooling

Optimisation 2

Reduce thread stack size or thread count.

Example

stack size.

1 MB $\rightarrow$ 512 kB

Q4 Starvation Debugging

Root cause

scheduler

Q1 $\rightarrow$ Interactive

Q2 $\rightarrow$ system

Q3 $\rightarrow$ Batch.

P5 is in Q3 (FCFS)

$\rightarrow$ Higher priority queues continuously receive new jobs.

therefore P5 never gets CPU time.

This is starvation.

Optimisation:

use Aging.

increase priority of waiting processes over time

Example

Every 5 seconds

priority ++

eventually P5 moves to Q2 or Q1

Result:-

- * fair scheduling

- * No starvation

- * All processes eventually execute.

5) context-switch overhead Debugging

Given

context switches = 20000/sec

Quantum = 1ms Average Burst = 2ms

High I/O frequency.

Root cause

The quantum is extremely small.

A 2ms job is preempted after only 1ms.

→ Each block/unblock generates more context switches

Optimisation

1) Increase quantum

ex: 1ms → 5ms.

2) use thread pooling and I/O completion mechanisms.

ex: event-driven I/O.

Async I/O

6) Producer-consumer Debugging

1. Synchronisation Error.

current producer.

wait(mutex)

wait(empty)

current consumer

wait(mutex)

wait(full).

→ Mutex is locked before checking resource availability.

→ Both wait forever.

Producer

wait(empty);

wait(mutex);

insert(item);

signal(mutex);

signal(full);

consumer

wait(full);

wait(mutex);

remove(item);

signal(mutex);

signal(empty);

Thread Performance Debugging

1. Thread Management Problem.
→ the server creates a new thread for every request.
15,000 request/sec produce enormous thread.
2. Why Performance Degrades.

Problems:

- continuous thread creation.
- * frequent context switching.
- * memory fragmentation.
- 3. Optimisations.

Thread Pool * create fixed worker threads.

Asynchronous / event-driven processing.

* epoll * Async I/O

→ I/OCP

Banker's Algorithm Debugging

Process

Max

Allocation

Need

P1

2 1 2

1 0 1

1 1 1

P2

1 2 1

0 1 0

1 1 1

P3

3 2 2

1 1 1

2 1 1

Need = Max - Allocation

Request ≤ Need

P2 1 1 1 ≤ 2 1 1

Available

2 1 1 - (1 1 1) = 1 0 0

P2 finishes first.

Safe sequence

P2 → P1 → P3

therefore request can be granted immediately.

because the system is in safe state.

P1 finishes

Available = 3, 2, 2

P3 finishes

safe sequence.

9) Deadlock Detection and Recovery Debugging

1. Deadlock Detection.

Graph.

$T_1 \rightarrow R_2 \rightarrow T_2$

$T_2 \rightarrow R_3 \rightarrow T_3$

$T_3 \rightarrow R_4 \rightarrow T_4$

$T_4 \rightarrow R_1 \rightarrow T_1$

cycle exists.

Deadlock exists.

Method 2

Resource preemption.

Rollback one transaction and release resources.

2. Detection method
The Resource Allocation Graph
is analysed.
A cycle involving all transaction
confirms deadlock.

10) Monitor-Based Synchronization Debugging

1. Logical error

the monitor uses

wait();

2. why jobs never execute

after

busy = false;

no thread is awakened.

3. correct monitor.

monitor printer

{ boolean busy = false;

condition printer_free;

procedure print()

{

while (busy)

wait(printer_free);

busy = true;

busy = false;

signal(printer_free);

}

}

}

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging(25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						
Q5						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____